

CO3-AT1-Analytical Problem Solving

Question 1: Banking Account Management System

A banking application needs to create and manage customer account objects. The system should support different ways of initializing an account depending on the information available.

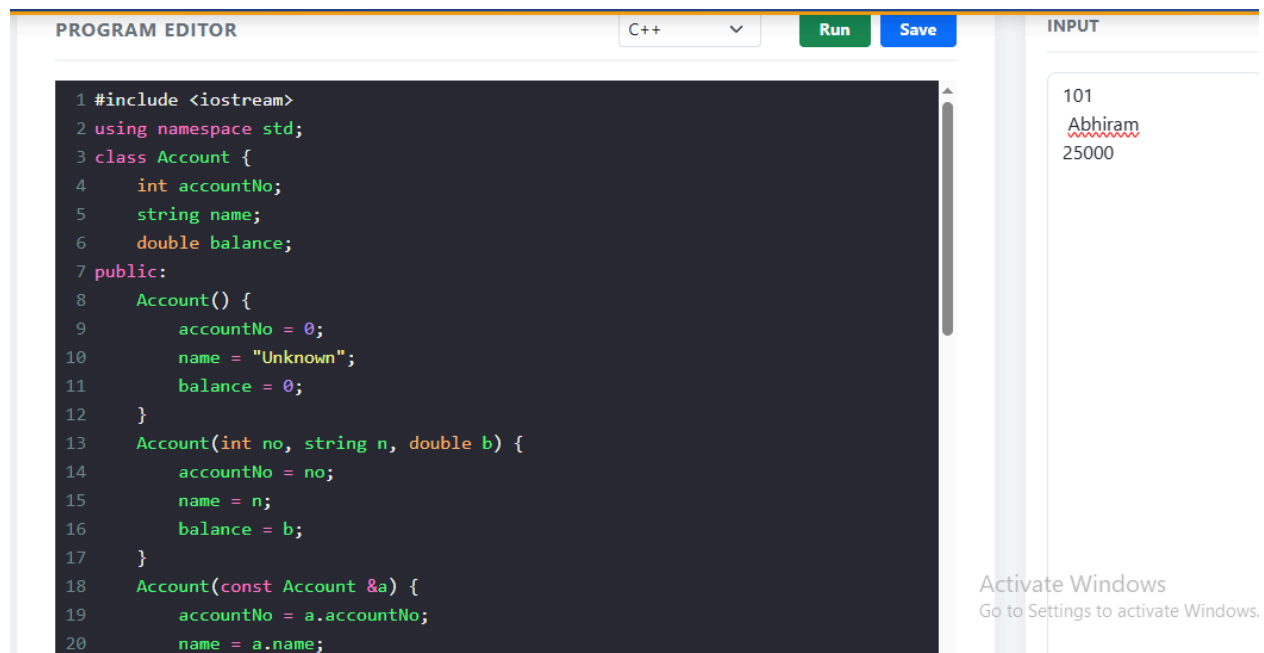
Design a C++ program using a class **Account** with the following constructors:

- Default constructor – initializes an account with default values.
- Parameterized constructor – initializes account number, customer name, and balance.
- Copy constructor – creates a new account by copying the details of an existing account.

Tasks:

1. Define the Account class with suitable data members.
2. Implement the default, parameterized, and copy constructors.
3. Create objects using each constructor.
4. Display the details of each account.
5. Analyze and explain how object initialization differs for the three constructors.
6. Create a new account object from an existing account object using the copy constructor.
7. Predict and explain the order in which constructors are called during object creation and copying.

Code:



The screenshot shows a C++ program editor with the following code:

```
1 #include <iostream>
2 using namespace std;
3 class Account {
4     int accountNo;
5     string name;
6     double balance;
7 public:
8     Account() {
9         accountNo = 0;
10        name = "Unknown";
11        balance = 0;
12    }
13    Account(int no, string n, double b) {
14        accountNo = no;
15        name = n;
16        balance = b;
17    }
18    Account(const Account &a) {
19        accountNo = a.accountNo;
20        name = a.name;
```

The output window on the right shows the following input:

```
101
Abhiram
25000
```

At the bottom right, there is a watermark that says "Activate Windows Go to Settings to activate Windows."

```

21     balance = a.balance;
22 }
23 void display() {
24     cout << "Account No: " << accountNo << endl;
25     cout << "Name: " << name << endl;
26     cout << "Balance: " << balance << endl;
27 }
28 };
29 int main() {
30     int no;
31     string name;
32     double balance;
33     Account a1;
34     cout << "Default Account:" << endl;
35     a1.display();
36     cout << "\nEnter Account Number: ";
37     cin >> no;
38     cout << "Enter Customer Name: ";
39     cin >> name;
40     cout << "Enter Balance: ";
41     cin >> balance;

```

```

101
Abhiram
25000

```

Activate Windows
Go to Settings to activate Windows.

```

42     Account a2(no, name, balance);
43     cout << "\nParameterized Account:" << endl;
44     a2.display();
45     Account a3(a2);
46     cout << "\nCopied Account:" << endl;
47     a3.display();
48     return 0;
49 }

```

OUTPUT

```

Default Account:
Account No: 0
Name: Unknown
Balance: 0

Enter Account Number: Enter Customer Name: Enter Balance:
Parameterized Account:
Account No: 101
Name: Abhiram
Balance: 25000

```

Question 2: Inventory Product Management System

A software company is developing an inventory management system using a class called **Product**. The system should allow products to be created, copied, combined, and removed from memory. Write a complete C++ program for the Inventory Product Management System using a parameterized constructor, copy constructor, destructor, and overloaded **+** operator. Demonstrate the complete object lifecycle through suitable output statements.

Analyze:

1. Design the Product class with appropriate data members.
2. Implement a parameterized constructor to initialize product details.
3. Implement a copy constructor to duplicate a product.
4. Implement a destructor to release resources when an object is destroyed.
5. Overload the **+** operator to add the stock quantities of two products.
6. Create two product objects using the parameterized constructor.
7. Create another product using the copy constructor.
8. Add two products using the overloaded **+** operator.
9. Display the product details and resulting stock quantity.
10. Trace and explain the sequence of constructor and destructor calls.

Code:

```
1 #include <iostream>
2 using namespace std;
3 class Product {
4     int id;
5     string name;
6     int stock;
7 public:
8     Product(int i, string n, int s) {
9         id = i;
10        name = n;
11        stock = s;
12    }
13    Product(const Product &p) {
14        id = p.id;
15        name = p.name;
16        stock = p.stock;
17    }
18    Product operator+(Product p) {
19        return Product(id, name, stock + p.stock);
20    }
21    void display() {
```

```
101
laptop
10
102
laptop
20
```

```

22     cout << "Product ID: " << id << endl;
23     cout << "Product Name: " << name << endl;
24     cout << "Stock: " << stock << endl;
25 }
26 ~Product() {
27     cout << "Destructor called for " << name << endl;
28 }
29 };
30 int main() {
31     int id1, id2, stock1, stock2;
32     string name1, name2;
33     cout << "Enter Product 1 ID: ";
34     cin >> id1;
35     cout << "Enter Product 1 Name: ";
36     cin >> name1;
37     cout << "Enter Product 1 Stock: ";
38     cin >> stock1;
39     cout << "\nEnter Product 2 ID: ";
40     cin >> id2;
41     cout << "Enter Product 2 Name: ";
42     cin >> name2;

```

```

101
laptop
10
102
laptop
20

```

```

48     Product p4 = p1 + p2;
49     cout << "\nProduct 1:" << endl;
50     p1.display();
51     cout << "\nProduct 2:" << endl;
52     p2.display();
53     cout << "\nCopied Product:" << endl;
54     p3.display();
55     cout << "\nCombined Product:" << endl;
56     p4.display();
57     return 0;
58 }

```

OUTPUT

```

Product 1:
Product ID: 101
Product Name: laptop
Stock: 10

Product 2:
Product ID: 102
Product Name: laptop
Stock: 20

```

Question 3: Shopping Cart Management System

An online shopping application maintains customer shopping carts. A cart contains multiple products and their quantities. The system should support creating a cart, duplicating an existing cart, and merging two carts. Write a complete C++ program to implement the `ShoppingCart` system.

Demonstrate parameterized construction, copy construction, cart merging using `+`, and destruction of objects.

Tasks:

1. Design the Product class with appropriate data members.
2. Implement a parameterized constructor to initialize product details.
3. Implement a copy constructor to duplicate a product.
4. Implement a destructor to release resources when an object is destroyed.
5. Overload the `+` operator to add the stock quantities of two products.
6. Create two product objects using the parameterized constructor.
7. Create another product using the copy constructor.
8. Add two products using the overloaded `+` operator.
9. Display the product details and resulting stock quantity.
10. Trace and explain the sequence of constructor and destructor calls.
11. Explain what may happen if dynamically allocated memory is not released in the destructor.

Code:

```
1 #include <iostream>
2 using namespace std;
3 class ShoppingCart {
4     int cartId;
5     string product;
6     int quantity;
7 public:
8     ShoppingCart(int id, string p, int q) {
9         cartId = id;
10        product = p;
11        quantity = q;
12    }
13    ShoppingCart(const ShoppingCart &c) {
14        cartId = c.cartId;
15        product = c.product;
16        quantity = c.quantity;
17    }
18    ShoppingCart operator+(ShoppingCart c) {
19        return ShoppingCart(cartId, product, quantity + c.quantity);
20    }
```

```
1
mobile
2
2
mobile
3
```

```

21 void display() {
22     cout << "Cart ID: " << cartId << endl;
23     cout << "Product: " << product << endl;
24     cout << "Quantity: " << quantity << endl;
25 }
26 ~ShoppingCart() {
27     cout << "Destructor called for Cart "
28         << cartId << endl;
29 }
30 };
31 int main() {
32     int id1, id2, q1, q2;
33     string p1, p2;
34     cout << "Enter Cart 1 ID: ";
35     cin >> id1;
36     cout << "Enter Product 1: ";
37     cin >> p1;
38     cout << "Enter Quantity 1: ";
39     cin >> q1;
40     cout << "\nEnter Cart 2 ID: ";
41     cin >> id2;

```

```

1
mobile
2
2
mobile
3

```

```

50     cout << "\nCart 1: " << endl;
51     c1.display();
52     cout << "\nCart 2:" << endl;
53     c2.display();
54     cout << "\nCopied Cart:" << endl;
55     c3.display();
56     cout << "\nMerged Cart:" << endl;
57     c4.display();
58     return 0;
59 }

```

OUTPUT

```

Cart 1:
Cart ID: 1
Product: mobile
Quantity: 2

Cart 2:
Cart ID: 2
Product: mobile
Quantity: 3

```